

Tarea Informe Procesamiento de texto

Julian Esteban Ballesteros Ortiz

Juan Diego Walteros Cortes

Especialización, Inteligencia Artificial

CAD2202023206: Procesamiento de Lenguaje Natural

Universidad de Cundinamarca

Profesor. Paul Alexander Diaz Montaña

domingo, 27 de septiembre de 2026

Resumen

Este informe documenta un laboratorio de procesamiento de lenguaje natural aplicado a 40 comentarios de demostración sobre un tutorial de Python. Se describen 17 procesos, desde normalización, análisis morfológico y sintáctico hasta representación TF-IDF, minería de texto, clasificación y tareas con modelos de lenguaje. El análisis combina las celdas ejecutadas del notebook con sus salidas registradas. En la partición de prueba de 12 comentarios, la regresión logística obtuvo accuracy de 0,58 y recall de 0,17 para la clase negativa cuando se eliminaron negaciones; al conservarlas, la accuracy registrada subió a 0,83 y el recall negativo a 0,67. La evidencia ilustra que el preprocesamiento depende de la tarea, aunque el tamaño y la composición del corpus impiden generalizar el resultado. Los comentarios positivos tuvieron más “likes” promedio que los negativos en esta muestra, asociación que no demuestra causalidad ni representa el conjunto de Kaggle. También se identificaron límites de ejecución: la celda de pregunta-respuesta no pudo cargar su modelo y usó recuperación TF-IDF; las salidas generativas muestran fluidez desigual. Se concluye que conviene combinar métodos interpretables y modelos neuronales con validación externa, preservación de evidencia y evaluación por tarea.

Palabras clave: procesamiento de lenguaje natural, TF-IDF, clasificación de texto, modelos de lenguaje, comentarios de YouTube

Procesamiento de texto para el análisis de comentarios de YouTube

GitHub

<https://github.com/Truder3105/procesamiento-de-texto-nlp>

Contenido del repositorio

```
procesamiento-de-texto-nlp/  
├── notebooks/  
│   └── Procesamiento_de_Texto_NLP.ipynb    ← Notebook principal (Google Colab)  
├── informe/  
│   └── Procesamiento_de_texto.pdf          ← Informe escrito final  
├── requirements.txt                       ← Dependencias del proyecto  
└── README.md
```

Introducción

El procesamiento de texto convierte lenguaje no estructurado en representaciones que pueden analizarse, compararse y utilizarse en modelos. Este laboratorio recorre etapas lingüísticas, estadísticas y neuronales con un caso común: comentarios sobre un video tutorial de Python. Su propósito es explicar qué hace cada técnica, cómo se implementó en el notebook y qué evidencia produjo, con atención a los límites del corpus y a la reproducibilidad.

El notebook fue diseñado para Google Colab y emplea NLTK, spaCy, scikit-learn, pandas, Sentence-Transformers y Transformers. Los modelos descargados de Hugging Face requieren conexión; varias celdas contemplan alternativas clásicas. Por ello, el informe distingue entre resultados observados y salidas de respaldo, y no trata todas las secciones como ejecuciones equivalentes.

Método y corpus

El conjunto de trabajo se construyó dentro del notebook como corpus de ejemplo inspirado en la estructura de YouTube Statistics de Kaggle (Patil, s. f.): 40 comentarios en español, 19 etiquetados como negativos y 21 como positivos, cada uno con una cantidad de likes. No es una extracción del dataset completo ni una muestra aleatoria de YouTube. La división de clasificación reservó 30 % de los ejemplos (12 comentarios; seis por clase) para prueba, con estratificación. El notebook contiene salidas guardadas; por tanto, los números aquí reportados corresponden a esas salidas y no a una nueva ejecución en este entorno (Ballesteros Ortiz & Walteros Cortes, 2026).

Las transformaciones se aplicaron al corpus con normalización, stopwords, stemming y lematización. spaCy procesó ejemplos dirigidos para POS, parsing y NER. CountVectorizer y TfidfVectorizer generaron representaciones dispersas; regresión logística y Naive Bayes sirvieron como clasificadores. NLTK (Bird et al., 2009), spaCy (Honnibal & Montani, 2017), scikit-learn (Pedregosa et al., 2011) y Hugging Face Transformers (Wolf et al., 2020) se utilizaron cuando las descargas funcionaron; las celdas implementaron respaldos basados en TF-IDF o diccionario cuando no pudieron cargar modelos.

El notebook `Procesamiento_de_Texto_NLP.ipynb` y los materiales del laboratorio se encuentran en el repositorio público <https://github.com/Truder3105/procesamiento-de-texto-nlp>.

Procesos de texto

Normalización de texto

La función de normalización convierte a minúsculas, elimina diacríticos y signos, y limpia elementos superficiales como URLs, menciones, espacios repetidos y repeticiones de caracteres. En la salida, “Excelente video...” pasa a una forma uniforme sin tildes ni puntuación. Esto reduce

variantes ortográficas para el conteo, pero puede borrar tono expresivo o distinciones; conviene conservar el texto original y parametrizar qué se elimina.

Código del notebook (fragmento)

```
df["texto_normalizado"] = df["comentario"].apply(normalizar_texto)
df[["comentario", "texto_normalizado"]].head(1)
```

Salida del notebook (extracto)

```
Excelente video, por fin entendí cómo funcionan las listas en Python!!!
-> excelente video por fin entendí como funcionan las listas en python
```

Análisis del código y su resultado. La función encadena reglas de limpieza y conserva el contenido léxico principal; el ejemplo queda en minúsculas y sin tildes ni signos. La salida confirma la transformación, aunque normalizar de esta forma también elimina señales expresivas.

Stopwords

NLTK devuelve 313 stopwords para español en la ejecución guardada. La función elimina palabras funcionales y tokens no alfabéticos; por ejemplo, “por”, “en” y “las” desaparecen. La configuración inicial también quitó “no” y “nada”, información decisiva para sentimiento. El propio notebook corrige esta lista preservando negaciones, mostrando que una lista genérica debe adaptarse a la tarea.

Código del notebook (fragmento)

```
stop_es = set(stopwords.words("spanish"))
def quitar_stopwords(texto):
    tokens = word_tokenize(texto, language="spanish")
    return [t for t in tokens if t not in stop_es and t.isalpha()]
```

Salida del notebook (extracto)

```
Total de stopwords en español (NLTK): 313
["fuiesteis", "estada", "y", "al", "tuvisteis", "suya", "estuvisteis", "estad", "mías",
"tuviste", ...]
```

Análisis del código y su resultado. El código tokeniza y filtra palabras funcionales y tokens no alfabéticos. La salida informa 313 elementos en la lista; como el filtro original quita también “no” y “nada”, el resultado puede invertir la señal de sentimiento.

Stemming

SnowballStemmer reduce formas por reglas morfológicas sin exigir que la raíz sea una palabra. “Explicando”, “explicación” y “explicado” se reducen a “explic”; “programación” a “program”. El procedimiento facilita agrupar variantes, pero su truncamiento sacrifica interpretabilidad y puede fusionar palabras distintas.

Código del notebook (fragmento)

```
stemmer = SnowballStemmer("spanish")
for palabra in ejemplos:
    print(f"{palabra:15s} -> {stemmer.stem(palabra)}")
```

Salida del notebook (extracto)

```
explicando      -> explic
explicación     -> explic
explicado       -> explic
```

Análisis del código y su resultado. El stemmer agrupa tres formas en una raíz común, útil para contar variantes. La raíz “explic” no es un lema español; la salida muestra por qué se gana agrupación a costa de legibilidad.

Lematización

spaCy aplica análisis morfológico y devuelve formas canónicas dependientes del contexto.

En los ejemplos, “explicando” se convierte en “explicar” y “corriendo” en “correr”; “explicación” se mantiene como sustantivo. Frente al stemming, el lema suele ser más legible, aunque la calidad depende del modelo lingüístico y del contexto.

Código del notebook (fragmento)

```
doc_ejemplo = nlp(" ".join(ejemplos))
for token in doc_ejemplo:
    print(token.text, "-> lema:", token.lemma_)
```

Salida del notebook (extracto)

```
explicando -> lema: explicar
explicación -> lema: explicación
explicado -> lema: explicado
```

Análisis del código y su resultado. spaCy procesa la secuencia con contexto morfológico. A diferencia del truncamiento anterior, devuelve formas interpretables y distingue el verbo “explicar” del sustantivo “explicación”.

Part-of-speech tagging

El etiquetador POS asigna categorías como sustantivo, verbo, adjetivo y adverbio. En la salida guardada, la extracción de adjetivos contabiliza “explicado” cuatro veces y “excelente” tres; también aparecen “horrible” y “rápido”. Son señales interpretables para opinión, pero limitarse a adjetivos omite negación, ironía y evaluaciones expresadas con verbos o sustantivos.

Código del notebook (fragmento)

```
doc = nlp("El video explica clases y objetos de forma clara")
for token in doc:
    print(token.text, token.pos_, token.tag_)
```

Salida del notebook (extracto)

TOKEN	POS	TAG
El	DET	DET
video	NOUN	NOUN
explica	VERB	VERB
clases	NOUN	NOUN

Análisis del código y su resultado. El bucle muestra la etiqueta universal y la etiqueta detallada por token. La secuencia identifica determinante, sustantivo y verbo de forma coherente; la extracción posterior de adjetivos ayuda a resumir opinión, pero no reemplaza el contexto.

Parsing

El análisis de dependencias conecta cada token con su cabeza sintáctica mediante relaciones gramaticales. En “El profesor explica muy bien los conceptos de programación”, “explica” se identifica como raíz, “profesor” como sujeto y “conceptos” como objeto. Estas relaciones permitirían extraer estructuras sujeto–acción–objeto; la precisión depende de la oración y del modelo spaCy.

Código del notebook (fragmento)

```
doc = nlp("El profesor explica muy bien los conceptos")
for token in doc:
    print(token.text, token.dep_, "->", token.head.text)
```

Salida del notebook (extracto)

```
profesor --nsubj--> explica
explica --ROOT --> explica
conceptos --obj --> explica
```

Análisis del código y su resultado. El código recorre relaciones de dependencia y muestra la cabeza de cada palabra. La salida identifica sujeto, raíz y objeto, lo que permite recuperar quién realiza la acción y sobre qué, siempre sujeto a errores del parser.

Named-entity recognition

NER identifica menciones de personas, organizaciones, lugares u otras clases. En la salida registrada, el modelo reconoce “Juan Pérez” como PER y “Universidad Nacional” como ORG, y etiqueta “Python” como MISC; no devuelve “CódigoFácil” ni “Google Colab” en esa ejecución. El modelo pequeño puede omitir o clasificar de forma inconsistente marcas y tecnologías, por lo que los conteos de entidades requieren revisión humana o evaluación contra anotaciones.

Código del notebook (fragmento)

```
doc = nlp(texto_ner)
for ent in doc.ents:
    print(ent.text, "->", ent.label_)
```

Salida del notebook (extracto)

```
Juan Pérez -> PER
Universidad Nacional -> ORG
Python -> MISC
```

Análisis del código y su resultado. El bucle entrega spans de entidad y su categoría. Los nombres propios explícitos se recuperan, mientras que la etiqueta MISC para una tecnología evidencia que las categorías del modelo pueden no coincidir con las clases de negocio deseadas.

Term frequency

La frecuencia de término (TF) cuenta ocurrencias dentro del documento y puede normalizarse por su longitud. En el vocabulario de 134 términos obtenido por CountVectorizer, los términos con mayor frecuencia guardada son “video” (8), “explicacion” (6), “contenido” (5), “ver” (4) y “explicado” (4). La frecuencia caracteriza el corpus, pero palabras comunes del tema pueden dominar sin ser discriminantes.

Código del notebook (fragmento)

```
count_vect = CountVectorizer()
tf_matrix = count_vect.fit_transform(corpus_lemas)
tf_df = pd.DataFrame(tf_matrix.toarray(),
                     columns=count_vect.get_feature_names_out())
```

Salida del notebook (extracto)

```
Vocabulario: 134 términos únicos
video 8
explicacion 6
contenido 5
ver 4
explicado 4
```

Análisis del código y su resultado. CountVectorizer construye una matriz documento-término de conteos. “video” domina con ocho ocurrencias; ese conteo caracteriza el corpus, pero no ajusta por longitud ni por cuán común sea el término entre documentos.

Inverse document frequency

IDF pondera la rareza de un término entre documentos; TF-IDF combina esa rareza con su frecuencia local. scikit-learn usa suavizado y normalización de vectores en TfidfVectorizer. En el comentario negativo seleccionado, “rápido”, “horrible” y “escuchar” reciben 0,478; “entendí” y “audio”, 0,396. Los pesos resaltan términos distintivos, no constituyen una explicación causal ni una medida de sentimiento.

Código del notebook (fragmento)

```
tfidf_vect = TfidfVectorizer()
tfidf_matrix = tfidf_vect.fit_transform(corpus_lemas)
idf_scores = pd.Series(tfidf_vect.idf_,
                      index=tfidf_vect.get_feature_names_out())
```

Salida del notebook (extracto)

```
IDF bajo: video 2.516347; explicacion 2.767662
Comentario negativo – mayor TF-IDF:
rapido 0.478; horrible 0.478; escuchar 0.478
```

Análisis del código y su resultado. El vectorizador calcula pesos que elevan términos raros y reducen términos extendidos. “Rápido” y “horrible” destacan en el comentario negativo, mientras que “video” tiene menor IDF; estos pesos son rasgos, no una explicación causal.

Text mining

La minería integra frecuencias, bigramas y variables estructuradas para explorar patrones. El notebook encuentra bigramas repetidos como “justo necesitar” y “perdí tiempo”. Al agrupar por etiqueta, los likes promedio son 10,05 en negativos ($n = 19$) y 319,52 en positivos ($n = 21$), cerca de 31,8 veces de diferencia. Es una asociación del corpus sintético de demostración; no demuestra que el sentimiento produzca likes y puede depender de cómo se construyeron las etiquetas y los valores.

Código del notebook (fragmento)

```
bigram_vect = CountVectorizer(ngram_range=(2, 2))
bigram_matrix = bigram_vect.fit_transform(corpus_lemas)
resumen = df.groupby("sentimiento")["likes"].agg(
    ["mean", "min", "max", "count"])
```

Salida del notebook (extracto)

```
Bigramas: "justo necesitar" (2), "perdi tiempo" (2)
Likes medios: negativo 10.05 (n=19); positivo 319.52 (n=21)
```

Análisis del código y su resultado. El código busca pares contiguos y agrupa los likes por etiqueta. Las medias difieren en esta muestra pequeña, pero el patrón no demuestra causalidad ni permite extrapolar al conjunto de Kaggle.

Sentence similarity

La similitud coseno compara vectores en un espacio común (Reimers & Gurevych, 2019). TF-IDF asigna 0,175 y 0,157 a dos comentarios positivos próximos al de referencia, pero también 0,150 a un comentario negativo que comparte la palabra “entendí”: la coincidencia léxica puede confundir polaridad. En la salida de embeddings multilingües sí se cargó el modelo y los vecinos principales fueron comentarios positivos con puntajes 0,383, 0,354 y 0,284. Los puntajes de ambas representaciones no deben compararse directamente como si estuvieran calibrados igual.

Código del notebook (fragmento)

```
sim_matrix = cosine_similarity(tfidf_matrix)
modelo = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
embeddings = modelo.encode(df["comentario"])
```

Salida del notebook (extracto)

```
TF-IDF: 0.175 Muy claro y conciso...; 0.157 Excelente aporte...; 0.150 No entendí nada...
Embeddings: 0.383 Muy buena explicación...; 0.354 Este canal se ha vuelto mi favorito...
```

Análisis del código y su resultado. La similitud coseno ordena comentarios por solapamiento ponderado. El tercer resultado comparte “entendí” pese a ser negativo; en contraste, los embeddings ejecutados devolvieron comentarios positivos como vecinos principales. Los puntajes provienen de espacios distintos y no se comparan como métricas calibradas.

Text classification

Se clasifican sentimientos positivo y negativo con TF-IDF, Naive Bayes y regresión logística. En la prueba de 12 ejemplos, Naive Bayes logró accuracy 0,67; regresión logística, 0,58. Para esta última, recall negativo fue 0,17 (1 de 6). Al retirar “no”, “nada” y “ni” de la lista de stopwords, la accuracy registrada subió a 0,83 y recall negativo a 0,67 (4 de 6); el F1 negativo aumentó de 0,29 a 0,80. El resultado apoya preservar negaciones, pero la muestra pequeña hace que una sola predicción cambie las métricas en 0,083; se requieren validación cruzada y datos externos.

Código del notebook (fragmento)

```
X_train, X_test, y_train, y_test = train_test_split(
    tfidf_matrix, y, test_size=0.3, random_state=42, stratify=y)
stop_es_corregido = stop_es - {"no", "nada", "ni"}
```

Salida del notebook (extracto)

```
Antes de conservar negaciones: accuracy 0.58; recall negativo 0.17
Después: accuracy 0.83; recall negativo 0.67; F1 negativo 0.80
```

Análisis del código y su resultado. El fragmento divide los datos de forma estratificada y conserva negaciones al construir las características. El cambio medido es una mejora en esta partición de 12 casos, no una estimación estable de rendimiento futuro.

Generación de texto usando LLM

Un modelo causal predice tokens siguientes condicionados al prompt. El notebook ejecutó datificate/gpt2-small-spanish; la salida produjo continuaciones en español, aunque con repeticiones y asociaciones débiles. Una salida fluida no garantiza exactitud factual: el modelo no

consulta una fuente de verdad y puede inventar contenido. Para un uso aplicado se necesitan límites de longitud, registro de prompts y revisión.

Código del notebook (fragmento)

```
generador = pipeline("text-generation", model="datificate/gpt2-small-spanish")
set_seed(42)
prompt = "Los modelos de lenguaje grande han cambiado la forma en que"
salida = generador(prompt, max_length=45, num_return_sequences=2)
```

Salida del notebook (extracto)

```
[Opción 1] Los modelos de lenguaje grande han cambiado la forma en que se escriben las
palabras.
Los lenguajes grandes han sido utilizados para describir las partes de una lengua o
una estructura de objetos...
```

Análisis del código y su resultado. La tubería carga un modelo causal y genera dos continuaciones con una semilla fijada. La salida empieza de forma gramatical, pero repite ideas y pierde coherencia; generar texto no equivale a verificar hechos.

Text generation y decodificación

El notebook compara greedy, beam search y sampling con temperatura. Greedy elige el token más probable; beam mantiene varias secuencias candidatas; sampling introduce variación. Las salidas muestran repetición en greedy, una continuación más conservadora con beam y mayor variabilidad con sampling. No hay una estrategia universalmente superior: la selección debe alinearse con diversidad, consistencia y tolerancia a errores.

Código del notebook (fragmento)

```
modelo.generate(entrada, do_sample=False) # greedy
modelo.generate(entrada, num_beams=5)    # beam
modelo.generate(entrada, do_sample=True,
               top_k=50, temperature=0.9) # sampling
```

Salida del notebook (extracto)

```
Greedy: "...se basa en la inteligencia artificial, que se basa..."
Beam: "El futuro de la inteligencia artificial es un tema de debate."
Sampling: continuación más variable
```

Análisis del código y su resultado. El código cambia la estrategia de decodificación. Las salidas muestran repetición en greedy y mayor variabilidad con sampling; la estrategia debe elegirse según la tarea y no garantiza por sí sola calidad.

Question answering

La pregunta del laboratorio pide identificar un problema descrito por usuarios. El modelo mrm8488/bert-spanish-cased-finetuned-squad2-es no pudo descargarse porque el identificador no se resolvió en Hugging Face. La celda ejecutó como respaldo recuperación de oración por TF-IDF

y devolvió la frase sobre problemas de calidad de audio. Esto es recuperación de evidencia, no una respuesta producida por el modelo QA pretendido; se debe validar el ID/modelo y medir exactitud de respuesta antes de atribuir rendimiento.

Código del notebook (fragmento)

```
qa_pipeline = pipeline("question-answering",  
    model="mrms8488/bert-spanish-cased-finetuned-squad2-es")  
resultado = qa_pipeline(question=pregunta, context=contexto)
```

Salida del notebook (extracto)

Aviso: el modelo no se pudo resolver en Hugging Face.
Alternativa TF-IDF: "...algunos usuarios señalan problemas de calidad de audio."

Análisis del código y su resultado. La llamada pretendía usar QA extractivo, pero el identificador del modelo falló. La salida pertenece al respaldo de recuperación por similitud de oraciones y no debe atribuirse a BERT-QA.

Summarization

Se implementa resumen extractivo con puntuaciones TF-IDF por oración y se intenta una alternativa abstractiva con BART. El extractivo devolvió oraciones del texto fuente sobre preentrenamiento y MLOps, lo que conserva literalidad pero puede ensamblar pasajes con poca continuidad. La salida abstractiva sí se generó, pero el texto guardado mezcla inglés y español ("trained with..."); no es evidencia de una traducción ni de resumen de calidad. Debe evaluarse cobertura, fidelidad y fluidez, por ejemplo mediante revisión humana.

Código del notebook (fragmento)

```
oraciones = sent_tokenize(texto_largo)  
puntajes = TfidfVectorizer(stop_words=stop_es).fit_transform(oraciones).sum(axis=1)  
resumen = " ".join(oraciones[i] for i in top_n_indices)
```

Salida del notebook (extracto)

RESUMEN EXTRACTIVO: "Gracias a este preentrenamiento a gran escala, aprenden patrones gramaticales, hechos del mundo..."

Análisis del código y su resultado. El método puntúa oraciones con TF-IDF y selecciona las más relevantes sin reescribirlas. La salida conserva frases de la fuente, aunque puede unir segmentos con una transición poco natural; el resumen abstractivo guardado mezcló inglés y español.

Translation

La celda ejecutó Helsinki-NLP/opus-mt-es-en y tradujo "Este video me ayudó..." como "This video helped me quickly understand the basics of programming." La salida conserva el sentido central con una formulación natural. La traducción automática puede omitir matices; una

única oración correcta no mide calidad general y debería complementarse con ejemplos diversos y revisión bilingüe.

Código del notebook (fragmento)

```
nombre_modelo_tr = "Helsinki-NLP/opus-mt-es-en"  
tokenizer_tr = AutoTokenizer.from_pretrained(nombre_modelo_tr)  
modelo_tr = AutoModelForSeq2SeqLM.from_pretrained(nombre_modelo_tr)  
outputs_tr = modelo_tr.generate(**tokenizer_tr(texto_es, return_tensors="pt"))
```

Salida del notebook (extracto)

```
ES: Este video me ayudó a entender rápidamente los conceptos básicos de programación.  
EN (LLM): This video helped me quickly understand the basics of programming.
```

Análisis del código y su resultado. El modelo seq2seq reordena palabras y produce una traducción idiomática. El ejemplo conserva el sentido principal, aunque una sola frase no permite estimar la calidad en distintos dominios.

Análisis integrado

Los resultados más sólidos para la enseñanza son los que pueden rastrearse a salidas numéricas o textuales guardadas: reducción morfológica, términos TF-IDF, clasificación antes/después de preservar negaciones, vecinos de similitud y estadísticas descriptivas de likes. En cambio, la QA no ejecutó el modelo designado, y una salida generativa o abstractiva no debe describirse como validada solo porque la celda terminó. Este registro de procedencia es parte de la interpretación técnica.

El patrón de sentimiento y likes es interesante para formular una hipótesis de producto, no para concluir que comentarios positivos causan interacción. El corpus fue construido manualmente, contiene solo 40 entradas y los resultados de clasificación se estiman en 12 casos de prueba; no hay muestreo aleatorio del conjunto Kaggle ni intervalos de confianza. Un estudio posterior debe recuperar datos con licencia y procedencia documentadas, separar entrenamiento/prueba por video para evitar fuga, fijar semillas, informar matrices de confusión y macro-F1, y evaluar el efecto de la popularidad previa, antigüedad y exposición del video.

La secuencia recomendada para un sistema de comentarios es conservar texto crudo; normalizar con reglas reversibles; mantener negaciones y marcadores de contraste; comparar TF-IDF con embeddings; evaluar clasificadores con particiones por video; inspeccionar errores y

sesgos; y registrar versiones de datos, dependencias, modelos y prompts. POS, parsing y NER pueden agregar explicabilidad estructurada, mientras que los LLM son adecuados para tareas generativas cuando se acompañan de verificación y controles de costo, latencia y privacidad.

Conclusiones

El laboratorio recorre los 17 procesos solicitados y enlaza limpieza, morfología, sintaxis, representación, minería, clasificación y tareas con LLM en un caso común. Los ejemplos muestran que cada representación responde a preguntas diferentes: TF-IDF facilita la interpretación léxica, embeddings aproximan similitud semántica y spaCy aporta estructura lingüística.

El resultado cuantitativo central es el cambio de la regresión logística al preservar negaciones: accuracy de 0,58 a 0,83 y recall negativo de 0,17 a 0,67 en una prueba pequeña. Esto demuestra que el preprocesamiento altera la señal disponible; no prueba que el modelo generalice. La asociación entre sentimiento positivo y más likes también debe tratarse como hipótesis exploratoria.

Antes de usar el flujo para predecir rendimiento de videos, el notebook debe ejecutarse de forma reproducible sobre datos reales, corregir el identificador de QA, evaluar salidas generativas con criterios explícitos y usar separación por video y métricas robustas. Estas medidas convertirían la demostración en evidencia evaluable y reducirían el riesgo de sobreinterpretar resultados.

Referencias

Advay Patil. (s. f.). YouTube statistics [Conjunto de datos]. Kaggle.

<https://www.kaggle.com/datasets/advaypatil/youtube-statistics>

Ballesteros Ortiz, J. E., & Walteros Cortes, J. D. (2026). Procesamiento de texto (NLP):

normalización, análisis lingüístico clásico y modelos de lenguaje [Notebook de Jupyter].

GitHub. <https://github.com/Truder3105/procesamiento-de-texto-nlp>

Bird, S., Klein, E., & Loper, E. (2009). Natural language processing with Python. O'Reilly Media.

Honnibal, M., & Montani, I. (2017). spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing. <https://spacy.io>

Hugging Face. (s. f.). Transformers documentation. <https://huggingface.co/docs/transformers>

Hugging Face. (s. f.). Helsinki-NLP/opus-mt-es-en [Modelo]. Hugging Face.

<https://huggingface.co/Helsinki-NLP/opus-mt-es-en>

NLTK Project. (s. f.). NLTK documentation. <https://www.nltk.org/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. <https://doi.org/10.18653/v1/D19-1410>

scikit-learn developers. (s. f.). Feature extraction: text feature extraction. https://scikit-learn.org/stable/modules/feature_extraction.html

spaCy. (s. f.). Linguistic features. <https://spacy.io/usage/linguistic-features>

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 38–45. <https://doi.org/10.18653/v1/2020.emnlp-demos.6>